

Magic2: Supplementary Technical Reference

Architecture, Algorithms, and Implementation

Jean Thierry-Mieg

Danielle Thierry-Mieg

Greg Boratyn

April 2026 — working draft

Contents

1	Overview and design philosophy	3
2	Hardware constraints and the sort–merge paradigm	3
2.1	Memory hierarchy and its consequences	3
2.2	The sort–merge alternative	3
3	Parallel execution: agents and channels	4
3.1	Channel library	4
3.2	Pipeline topology	4
3.3	NUMA-aware execution	4
4	Seed generation and genome indexing	5
4.1	Seed encoding	5
4.2	Strand-agnostic seed generation	5
4.3	Paired-read encoding	5
4.4	Subsampling	5
4.5	Target multiplicity filtering and jump encoding	6
4.6	Junction seeds from annotated splice sites	6
5	Sorting, joining and optional GPU acceleration	6
5.1	Data structures	6
6	Radix sort	7
6.1	Genome index loading and joining	7
6.2	Optional GPU acceleration	7
6.3	Latency hiding through channel-based concurrency	7
6.4	Per-block GPU pipeline	8
7	Alignment extension	9
8	Intron boundary optimisation, poly(A) detection, and splice-leader identification	9
9	Output formats	9
10	Quality-control metrics	10

11 Automatic parameter selection	10
12 Build system and supported platforms	10

1 Overview and design philosophy

Magic2 is a complete rewrite and extension of the earlier Magic-BLAST aligner, developed at the National Library of Medicine (NLM), NIH. It aligns short (Illumina) and long (Nanopore, PacBio) RNA-seq or DNA-seq reads against an annotated or unannotated reference genome, and in a single pass produces alignments, gene-expression counts, intron-support tables, poly(A) addition sites, coverage tracks, and QC metrics.

A posteriori, the name Magic2 can be parsed as *Memory-Aware Genomic Integrative Computation* — an acronym that reflects the central role of memory efficiency in the design.

Three principles govern all architectural decisions:

1. **Cache efficiency.** The dominant cost in large-scale genomic computations is data movements, not arithmetics. All core algorithms are designed to generate long sequential memory-access patterns rather than random accesses, keeping working data inside CPU caches and saturating memory bandwidth.
2. **Automatic configuration.** Read length, paired-end geometry, adapter sequences, strandedness, and sequencing platform are inferred from the data. Internal parameters (seed length, maximum intron size, subsampling density) are derived from genome size and reported to the user for transparency. Manual overrides remain possible but are rarely necessary.
3. **Single-pass richness.** All integrative outputs — alignments, expression counts, splice-junction tables, coverage tracks, poly(A) calls, QC metrics — are produced during the alignment pass while reads and seeds reside in memory, avoiding the need for secondary tools.

The source code is available at <https://github.com/NLM-DIR/Magic2> (branch `master`). The main entry point is `wsa/sa.main.c`; the central header defining all structures and function prototypes is `wsa/sa.h`.

2 Hardware constraints and the sort–merge paradigm

2.1 Memory hierarchy and its consequences

Modern CPUs execute each elementary operation in approximately one nanosecond when the operands reside in L1 or L2 cache, but fetching the same data from main RAM typically costs 60–100 ns on current x86 processors. A page fault to disk costs milliseconds. Because large genomic datasets far exceed cache capacity, algorithm design must prioritise access patterns over theoretical operation counts.

The core challenge in alignment is *pre-assignment*: given a short read, rapidly identify the small number of genomic loci where alignment is likely. Hash tables offer $O(1)$ average lookup, but on a multi-gigabyte genome index each lookup jumps to a quasi-random memory address, triggering cache misses and page faults. Measured latency per seed commonly reaches 100–300 ns, turning a theoretically fast algorithm into a memory-bandwidth bottleneck.

2.2 The sort–merge alternative

Magic2 adopts a *sort–merge* paradigm:

1. Generate seed records for reads and genome as large, contiguous blocks written sequentially to memory.
2. Sort both lists by seed value using a hard-coded cache-efficient radix sort (Section 6).
3. Join the two sorted lists by simultaneous linear traversal: both pointers advance monotonically, so the entire join operates strongly favors CPU cache and pre-fetching.

This approach is not novel in principle; it underlies high-performance database engines and has been applied in bioinformatics, notably in the DIAMOND protein aligner (?). Its roots lie in classical external sorting and merge-join algorithms (?), with modern refinements in cache-oblivious algorithms (?).

The key trade-off is replacing $O(n)$ cache hostile random hash look ups by two cache friendly operations: $O(n \log n)$ sorting followed by $O(n)$ sequential merging. The merge step is so cache-friendly that it runs at near-memory-bandwidth speed, far outpacing hash-table approaches on large genomes.

If available, the whole preassignment is automatically offloaded to a GPU (Section 5).

3 Parallel execution: agents and channels

3.1 Channel library

Magic2 exploits multiple cores through a clean, lock-free framework in which independent worker threads (*agents*) communicate solely through *channels* — bounded, self-synchronising queues implemented in `wchannels/channel.c`, closely following the paradigm of Go channels.

A channel is typed and has a fixed capacity. An agent inserting into a full channel blocks until space is available; an agent extracting from an empty channel blocks until data arrives. This built-in backpressure eliminates explicit locks and condition variables within agents, greatly simplifying both implementation and reasoning about correctness.

Channels track the number of active producers. When all producers have finished and decremented the source count to zero, the channel closes automatically; consumers drain any remaining records and then exit cleanly. This guarantee — no data loss and deadlock-free termination — holds regardless of the number of agents or the order in which blocks are processed.

3.2 Pipeline topology

The master thread enqueues descriptors of independent data blocks containing up to `bMax` Mega-bases (default 3 Mb) into a shared input channel. A pool of agents retrieves blocks from this channel, executes the complete per-block pipeline (seed identification, alignment extension, adaptor detection, splicing, scoring mismatches, pairing), and forwards results to designated output channels. Dedicated consumer agents drain output channels to merge the per-chromosome wiggle profiles, cumulate the gene expression counts, the introns, the poly-A sites and compile QC tables.

Each agent follows a straightforward loop:

```
while ((block = channelGet(inputChannel)) != NULL) {
    processBlock(block);
    channelPut(outputChannel, result);
}
signalDone(outputChannel);
```

Agents never wait on one another; all synchronisation emerges from channel blocking and closure. The main program defines the pipeline topology by creating channels, setting source counts, launching agents, and seeding the input channel but neither the main program nor the agents contain any explicit calls to the low level unix semaphores and other synchronization tools.

3.3 NUMA-aware execution

During its initialisation, Magic2 analyses the hardware and counts the number of CPUs, nodes and eventual GPU. Then Magic2 re-spawns itself and binds to the least-loaded node, and eventually the least busy GPU, minimising cross-node memory traffic. Users do not need set a thread count; Magic2 manages all concurrency

automatically and continuously monitors its usage of memory by varying the number of datablocks processed in parallel.

For debugging, `--debug` forces fully serial execution (`nAgents=1`, `nBlocks=1`) with verbose output. `--numactl` suppresses re-spawning while retaining full parallelism.

4 Seed generation and genome indexing

4.1 Seed encoding

Magic2 uses exact k -mer seeds of length $k \leq 19$, encoded as 32-bit unsigned integers (2 bits per base, so 16 bases fill exactly 32 bits). The default k is chosen automatically from genome size:

- $k = 16$ for small genomes (< 1 Gb; e.g., *C. elegans*, *D. melanogaster*);
- $k = 18$ for large genomes (human, mouse).

When $k > 16$, the extra bases are handled by radix partitioning into 16 or 32 separate seed tables indexed by the least significant base pair(s), providing effective 18- or 19-mer specificity while keeping each individual table within 32 bits.

Continuous (ungapped) k -mers are preferred over spaced seeds: spaced seeds are counterproductive for indel-prone technologies (Nanopore, PacBio) because indels in the spacers disrupt the spaced-seeds.

4.2 Strand-agnostic seed generation

To halve index size and enable strand-agnostic matching, both the forward k -mer S and its reverse complement C are computed simultaneously using an incremental rolling update:

$$S_{\text{new}} = ((S \ll 2) | b) \& \text{mask} \quad (1)$$

$$C_{\text{new}} = ((C \gg 2) | (b' \ll (k - 2))) \& \text{mask} \quad (2)$$

where b is the next base (2-bit coded) and b' its complement. The strand-agnostic value $Z = \min(S, C)$ is retained. The coordinate is encoded as $(x \ll 1)$ if S was chosen (forward strand) or $(x \ll 1 | 1)$ if C was chosen (reverse strand). Ambiguous bases reset the rolling accumulator; a seed is emitted only after k consecutive unambiguous bases.

4.3 Paired-read encoding

Paired-end reads arrive with names of the form `xx/1` and `xx/2`. Only the base name `xx` is entered into the read dictionary (`dictAdd(dict, "xx", &n)`), yielding a compact integer n . Then mate 1 and mate 2 are stored as contiguous numbers $2n$ and $(2n + 1)$ such that sorting by reads automatically implies sorting by pairs. The alignment stage therefore receives tables in which read-pair processing requires no further regrouping.

4.4 Subsampling

To reduce index size without sacrificing alignment sensitivity, seeds are sub-sampled using a *rolling-min-hash*: a round-robin buffer of size p (default 4) stores the hash of Z at recent positions; only the position with the minimum hash among the last p values is retained as a seed. This guarantees:

- average inter-seed spacing of $p/2$ positions;

- maximum gap of p positions;
- near-perfect synchronisation between read and genome seeds (in practice alignment begins by the second seed even when $p \leq 8$).

4.5 Target multiplicity filtering and jump encoding

After collection, target seeds are sorted (Section 6) and their multiplicity (number of occurrences in per target) is counted. Seeds with multiplicity $> Q$ (default 81) are discarded to limit spurious hits from repetitive elements; empirical tests showed $Q = 31$ gives higher specificity while $Q = 81$ slightly reduces false negatives at acceptable cost in index size and runtime.

Jump information is then embedded directly in the seed records: every 256th seed stores pointers to future seeds at distances $n_1 \times 256$, $n_2 \times 256$, $n_4 \times 256$, enabling fast skip-ahead during the merge step.

The filtered, jump-augmented target table is written to disk in compact binary format for later memory-mapped access during production alignment. Seed records are 16 bytes each (Section 5.1). overhead.

4.6 Junction seeds from annotated splice sites

When a GFF/GTF annotation is supplied, Magic2 supplements genomic seeds with *junction seeds* spanning annotated exon–exon boundaries, improving detection of short exons with known junctions.

For each annotated mRNA, all exons are concatenated *in silico* to form a continuous spliced transcript. All k -mers straddling each junction with at least 4 bases on each side ($k - 4$ starting positions per junction, with $p = 1$ so every valid position is used) are generated. For each such k -mer, two seed records with identical value Z are emitted:

- one pointing to the donor site (end of upstream exon);
- one pointing to the acceptor site (start of downstream exon), augmented with ancillary phase-shift information encoding the exact junction offset.

This dual-record strategy ensures that a single seed match near a junction can trigger extension on both sides of the splice site.

The additional index size is negligible: with $k = 18$ and $\sim 300,000$ annotated human introns, $(k - 4) \times 300,000$ yields only a few million additional seeds, compared to ~ 1 billion genomic seeds. Junction seeds are merged with the genomic table, sorted, and stored in the same binary format.

5 Sorting, joining and optional GPU acceleration

5.1 Data structures

Seed records are `CW` structs (16 bytes, 16-byte aligned), declared in `wsa/sa.common.h`:

Field	Type	Meaning
<code>seed</code>	<code>unsigned int</code>	32-bit 16-mer seed
<code>nam</code>	<code>unsigned int</code>	read or chrom identifier
<code>pos</code>	<code>unsigned int</code>	coordinate of first base of seed
<code>flags</code>	<code>unsigned int</code>	auxiliary information

The 16-byte size is deliberate: four `CW` records fill one 64-byte cache line exactly, which is optimal for both the CPU radix sort and GPU coalesced memory access.

6 Radix sort

[PLACEHOLDER — to be drafted in the next session using `wsa/sa.sort.c` as reference.]

Key points to cover:

- 3-pass radix sort implementation (April 2026 optimisation).
- Cache efficiency: why radix sort outperforms comparison-based sorts on this data.
- GPU offload via CUDA Thrust (optional, silent fallback to CPU); see Section 5.
- Benchmark: 15 % reduction in elapsed and CPU time on large datasets observed after optimisation.
- Dedicated sort–merge step for combining read and genome seed tables.

6.1 Genome index loading and joining

The filtered genome index (Section 4) is partitioned into NN sub-tables (16 or 32, indexed by the least-significant base pair(s) of the seed value). At the start of a run, all partitions are memory mapped. The read seeds and genome seed records are identical.

Both tables are scanned in parallel, advancing the pointer on the side of the lower seed. When there is a coincidence, the local Cartesian product of all entries with the matching seed is emitted as 64 bytes SEEDMATCH structs, by simply joining the 2 matching records. This table is then sorted by read (hence automatically by pairs).

6.2 Optional GPU acceleration

When an NVIDIA GPU is available, Magic2 offloads three steps of the seed-matching pipeline to the device via CUDA Thrust (`wsa/sa.gpusort.cu`): sorting the read-seed table, and joining it against the pre-loaded genome index, then sorting the table of SEEDMATCH.

GPU use is activated at compile time (`-DUSEGPU`) and detected at run time; execution falls back silently to the CPU radix sort (Section 6) when no compatible device is present. If a GPU is present, the genome seed-index is transferred to the GPU prior to the calculation of the first data block and discarded from the CPU side.

All structure declarations are shared and since both the CPU and the GPU use 64-bytes aligned buffer, no data conversion is required at the CPU–GPU boundary.

6.3 Latency hiding through channel-based concurrency

The GPU call in each agent is synchronous: the agent blocks on the device until the sorted SEEDMATCH table is ready, then resumes. No asynchronous CUDA streams, double-buffering, or explicit staging logic are required.

The standard GPU programming approach to this problem uses CUDA streams with asynchronous transfers (`cudaMemcpyAsync`): while the device processes block N , the host prepares and begins transferring block $N + 1$ into a second buffer, so that the PCIe transfer latency of one block is overlapped with the GPU computation of the next. This *double-buffering* pattern is effective but requires the programmer to manage two buffer sets, carefully sequence stream operations, and reason explicitly about which synchronisation points are safe — a non-trivial engineering burden that is easy to get wrong silently.

Magic2 avoids this complexity entirely. The agent pool, coordinated by bounded blocking channels, provides CPU–GPU overlap as a natural consequence of its concurrency model. While one agent is blocked

waiting for the GPU, all other agents continue executing their own blocks on CPU — extracting seeds, extending alignments, writing coverage tracks, and accumulating junction and QC tables. The blocked agent simply parks at what is, from the pipeline’s perspective, an ordinary blocking-channel operation, indistinguishable from blocking on an empty input queue. Provided the number of active agents is sufficient to keep both the GPU and the CPU cores occupied — a condition that holds automatically given Magic2’s block-size and agent-count defaults — GPU transfer and sort latency is fully overlapped with productive CPU work on other blocks, with no GPU-specific orchestration code in the pipeline at all.

The channel library (`wchannels/channel.c`) is a direct C implementation of Go’s channel semantics (?), themselves rooted in Hoare’s Communicating Sequential Processes (?). It was written to bring Go’s concurrency model to an existing 700,000-line C codebase that could not be migrated to Go. Go programmers obtain the same GPU-latency-hiding property for free from the goroutine scheduler, which parks a goroutine blocked on a channel and runs another in its place. Magic2’s agent/channel architecture replicates this behaviour in C, without any GPU-specific orchestration code in the pipeline.

6.4 Per-block GPU pipeline

Magic2’s alignment pipeline processes independent data blocks of up to `bMax Mb` through a pool of agents coordinated by channels (Section 3). GPU acceleration is applied at the block level: each agent, upon receiving a block, calls the GPU for seed sorting and joining, then proceeds to CPU-side alignment extension once the device returns. Within each block the GPU executes the following steps.

1. **Read-seed sort.** For each of the NN seed partitions of the current block, the `CW` array produced by CPU seed extraction is copied to device memory and sorted by seed value using `thrust::sort` with a device comparator that orders records by the `seed` field.
2. **Run-length encoding.** `thrust::reduce_by_key` compresses both the (resident) genome partition and the (freshly sorted) read partition into compact vectors of unique seed values with their start offsets and run lengths. This step replaces the linear merge traversal used in the CPU sort–merge path with a fully parallel reduction.
3. **Common-seed identification.** `thrust::set_intersection` finds seed values present in both the genome partition and the read-block partition. `thrust::lower_bound` then locates each common seed in the respective run-length tables, yielding the start offset and count for each matching group in both the index and the reads.
4. **Cartesian-product emission.** For each common seed value with m genomic occurrences and n read occurrences, the CUDA kernel `EmitCartesianProduct` writes $m \times n$ `SEEDMATCH` records to a pre-allocated output buffer. The kernel is launched with one thread block per common seed and up to 256 threads per block; each thread handles one or more pairs $(i_{\text{genome}}, j_{\text{read}})$ from the Cartesian product. Output positions are determined by an exclusive prefix scan (`thrust::exclusive_scan`) over per-seed pair counts, ensuring contiguous, conflict-free writes with no atomic operations.
5. **Sort by read pair.** The completed `SEEDMATCH` table is sorted on the GPU by the `read` name field using `thrust::sort`. Because of the paired-read encoding (Section 4.3), mate 1 ($n \ll 1$) and mate 2 ($(n \ll 1) \mid 1$) of each pair differ only in the least-significant bit of the `nam` field; sorting by this value simultaneously groups all candidate hits by read pair and places mate 1 immediately before mate 2 within each pair. This final GPU sort delivers a table that is directly consumable by the CPU alignment stage with no further reordering.

6. **Return to CPU.** The sorted `SEEDMATCH` table is copied back to host memory. The CPU extension stage (Section 7) processes one read pair at a time, extending each candidate seed match into a full alignment and performing splice-junction analysis, without any further access to the genome index.

7 Alignment extension

[PLACEHOLDER — to be drafted in a future session using `wsa/sa.align.c` as reference.]

Key points to cover:

- Banded Smith–Waterman extension from seed anchors.
- Error cost model (`errCost=4`, `errRateMax=12 %`).
- Handling of mismatches vs. indels for short vs. long reads.
- `minAli=30`, `minScore=30` thresholds.
- Multi-mapping and `maxTargetRepeats=81`.

8 Intron boundary optimisation, poly(A) detection, and splice-leader identification

[PLACEHOLDER — to be drafted in a future session using `wsa/sa.introns.c` as reference.]

Key points to cover:

- Splice-signal scoring (GT-AG, AT-AC, non-canonical).
- `maxIntron=1,000,000` bases.
- Poly(A)/poly(T) tail detection algorithm.
- Trans-spliced leader (SL) detection in *C. elegans*.
- Strand inference from splice signals.

9 Output formats

[PLACEHOLDER — to be drafted in a future session.]

Key points to cover:

- BAM output (`wsa/sa.sam.c`): default; standard downstream compatibility.
- `.hits` compact format (`wsa/sa.tabular.c`): available via `--hits`; faster I/O for internal pipelines.
- TSF (Tag–Sample Format): tab-separated, columns are `run/tag/format-code/data`; format codes `i`=integer, `f`=float, `t`=text; files need not be sorted; directly concatenable for merging; can produce 2-D tables compatible with Excel via `tsf.c`.
- Wiggle/coverage tracks (`wsa/sa.wiggle.c`): strand-specific; read-start/end profiles for TSS/TES detection.
- QC metrics table (`wsa/sa.stats.c`).

10 Quality-control metrics

[PLACEHOLDER — to be drafted in a future session using `wsa/sa.stats.c` as reference.]

Key points to cover:

- Mapping-class fractions: CDS, UTR, intron, intergenic, mitochondrial.
- Strand-specificity score.
- Concordant-pair rate; insert-size distribution.
- Mismatch distribution (per-cycle and global).
- Perfect-alignment count (zero-mismatch, zero-indel reads).
- Adapter detection and trimming statistics.
- All metrics emitted in TSF format; directly comparable across runs.

11 Automatic parameter selection

[PLACEHOLDER — to be drafted in a future session using `wsa/sa.config.c` as reference.]

Confirmed defaults (April 2026):

Parameter	Default
<code>errCost</code>	4
<code>errRateMax</code>	12 %
<code>bMax</code>	3 Mb per block
Index step	6 (genomes > 1 Mb)
Alignment step (<code>iStep</code>)	<code>tStep/2</code>
<code>seedLength</code>	16 (genomes < 1 Gb); 18 (human/mouse)
<code>maxTargetRepeats</code>	81
<code>minAli</code>	30
<code>minScore</code>	30
<code>maxIntron</code>	1,000,000 bases

12 Build system and supported platforms

Magic2 is written in C and built with `make` and `tcsh`. The repository lives inside the global `acedb` distribution (~700,000 lines of C); Magic2 depends on `acedb` libraries (notably `w1`, `wego`) and has not yet been fully separated.

Setting	Value
Optimised Linux build flag	<code>ACEDB_MACHINE=LINUX_4_OPT</code>
64-bit variant	<code>ACEDB_MACHINE=LINUX_X86_64_4_OPT</code>
Output binary	<code>Magic2/bin.LINUX_4_OPT/magic2</code>
Primary platform	Linux x86-64
Secondary platforms	Linux ARM, macOS
Optional GPU acceleration	CUDA (Thrust); silent CPU fallback

Continuous integration currently uses Travis CI (`.travis.yml`). Migration to GitHub Actions with matrix builds for all three target platforms is planned.